

PERSONAL LEARNING EDITION • PYTHON 3

AI PERSONAL STUDY ASSISTANT

PYTHON PROGRAMMING

Learn the basics. Practise with purpose. Build real projects.

STUDENT NAME
MOHAMMED NAHIYAN
DEPARTMENT
COMPUTER SCIENCE(AIML)

Inside this guide • Study notes • Code examples • Practice quiz • Mini-projects • 4-week plan

Tip: You can fill in these fields digitally using a PDF editor, or print this page and write your details by hand.

Welcome to your Python learning journey

This guide is designed for beginners learning Python 3. Work through one section at a time. Type the examples yourself, predict the output before running them, and change the values to explore what happens.

Your AI tutor prompt

Copy this into an AI assistant whenever you study: *“Act as my Python personal tutor. Teach one concept at a time in simple language. Ask what I already know, explain with a small example, quiz me before revealing answers, review my code kindly, and give me a short practice task. Give hints before full solutions.”*

Learning roadmap

1. Learn syntax, variables, types, and input/output.
2. Practise conditions, loops, strings, and collections.
3. Write functions and handle common errors.
4. Work with files, modules, and classes.
5. Build projects, test them, and explain your code.

Setup

- Install Python 3 from <https://www.python.org/downloads/> or use a browser-based Python editor.
- Use IDLE or an editor such as Visual Studio Code.
- Save Python programs with the .py extension.
- Errors are part of learning—read them carefully and try again.

Your first program

```
print("Hello, Python!")
```

print() displays information. Text goes inside quotes. Python is case-sensitive: print and Print are different names.

1. Variables and data types

```
student_name =  
"Amina" age = 18  
score = 92.5  
is_enrolled = True  
print(student_name, age, score, is_enrolled)
```

TYPE	EXAMPLE	COMMON USE
int	42	Whole numbers
float	3.14	Decimal numbers
str	"hello"	Text
bool	True / False	Logical values
NoneType	None	No value / missing value

```
price = 19.99  
quantity = 3  
total = price * quantity
```

A variable is a name that refers to a value. Choose clear names such as student_name, score, or total_price. Python conventionally uses snake_case.

```
print(total) # 59.97
```

Use `type(value)` to inspect a value's type. Comments begin with `#` and are ignored when the program runs.

2. Operators and user input

Operators

Arithmetic operators include `+`, `-`, `*`, `/`, `//` (floor division), `%` (remainder), and `**` (power). Comparisons include `==`, `!=`, `<`, `<=`, `>`, `>=`. Logical operators are `and`, `or`, and `not`.

```
print(7 / 2) # 3.5
print(7 // 2) # 3
print(7 % 2) # 1
print(2 ** 3) # 8
```

Input

```
name = input("What is your name? ")
print("Welcome,", name)
age = int(input("Age: "))
print("Next year:", age + 1)
```

`input()` returns text. Convert it using `int()` or `float()` when you need a number. Invalid conversions can raise `ValueError`.

3. Conditions and loops

if / elif / else

```
temperature = 28
if temperature >= 30:
    print("Hot")
elif temperature >= 20:
    print("Comfortable")
else:
    print("Cool")
```

Indentation defines code blocks. Four spaces per indentation level is the common convention.

for and range

```
for number in range(1, 6):
    print(number)
```

This prints 1 through 5 because the stop value in `range()` is excluded.

while

```
count = 3
while count > 0:
    print(count)
    count -= 1
print("Go!")
```

- `break` exits the nearest loop.
- `continue` skips to the next iteration.
- Make sure a while loop can eventually become false.

```
total = 0
for number in range(1, 6):
    total += number
print(total) # 15
```

4. Core collections

Lists

```
fruits = ["apple", "banana", "mango"]
fruits.append("orange")
print(fruits[0]) # apple
print(len(fruits))
```

Lists are ordered and mutable. Indexing starts at zero; -1 refers to the last item.

Tuples and sets

```
point = (4, 7) # tuple: ordered, immutable
unique_numbers = {1, 2, 2, 3} # set: unique values
print(point[0])
print(unique_numbers)
```

Dictionaries

```
student = {"name": "Amina", "mark": 92}
print(student["name"])
student["mark"] = 95
print(student.get("email", "Not provided"))
```

A dictionary maps keys to values. Use .get() if a key might be missing.

- List: ordered items that may change.
- Tuple: ordered items intended to stay fixed.
- Set: unique items.
- Dictionary: key-value pairs.

5. Functions

Functions package reusable logic. Define them with def, use parameters for inputs, and return to send a result back.

```
def calculate_area(width, height):
    return width * height

print(calculate_area(5, 3)) # 15
```

Default arguments and good habits

```
def greet(name, greeting="Hello"):
    return f"{greeting}, {name}!"

print(greet("Sam"))
```

- Use meaningful names.
- Keep each function focused on one task.
- Prefer parameters and return values over changing global variables.
- Functions make programs easier to test and reuse.

6. Strings and useful built-ins

```
message = " Learn Python "
print(message.strip())
print(message.lower())
print(message.replace("Python",
"coding")) name = "Ravi"
score = 87
print(f"{name} scored {score} marks.")
```

Strings are immutable: string methods return new strings. Helpful built-ins include len(), sum(), min(), max(), sorted(), enumerate(), and zip().

```
names = ["Asha", "Ravi", "Noor"]
for index, name in enumerate(names, start=1):
    print(index, name)
```

7. Errors, debugging, and exceptions

Common errors include `SyntaxError` (invalid syntax), `NameError` (unknown name), `TypeError` (unsupported operation for a type), and `ValueError` (invalid value). Read the error message and traceback carefully.

```
try:
    age = int(input("Enter age:
")) except ValueError:
    print("Please enter a whole number.")
else:
    print("Age entered:", age)
```

Debugging checklist

1. Read the error and line number.
2. Reproduce the issue with a small example.
3. Inspect intermediate values.
4. Check spelling, indentation, data types, and edge cases.
5. Change one thing at a time and run again.

8. Files and modules

Read and write a text file

```
with open("notes.txt", "w", encoding="utf-8") as file:
    file.write("Study Python every day.\n")

with open("notes.txt", "r", encoding="utf-8") as file:
    content = file.read()
    print(content)
```

The `with` statement closes the file reliably. Mode `'w'` replaces content, `'a'` appends, and `'r'` reads.

Modules

```
import math
print(math.sqrt(81))
from random import randint
print(randint(1, 6))
```

Modules provide reusable code. Python's standard library includes `math`, `random`, `dates`, `JSON`, and many other tools.

9. Object-oriented programming

Object-oriented programming (OOP) groups data and behaviour into classes and objects.

```
class Dog:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def describe(self):
        return f"{self.name} is {self.age} years old."

pet = Dog("Milo", 3)
print(pet.describe())
```

- Class: a blueprint for objects.

- Object: an instance of a class.
- Attribute: data stored on an object.
- Method: a function defined inside a class.
- Inheritance: a class reuses or extends another class.

10. Mini-projects

Project A — Number guessing game

Use `random.randint()` to choose a number. Let the player guess in a loop. Print “too high” or “too low”, count attempts, and allow quitting.

Project B — Quiz app

Create question-answer pairs, ask each question, compare answers with `.strip().lower()`, and keep a score.

Project C — To-do list

Store tasks in a list and create a menu to add, show, and remove tasks. Stretch goal: save tasks to a file.

Project D — Expense tracker

Store descriptions and amounts in dictionaries, list each expense, and calculate the total. Add validation for amounts.

Project checklist

- Does it handle empty or invalid input?
- Are names clear?
- Have you tested at least three cases?
- Can you explain each line in your own words?

11. Practice quiz

1. What does `print()` do?
2. What is the difference between `=` and `==`?
3. What is the output of `10 // 3`?
4. Why does `range(1, 4)` exclude 4?
5. Which collection stores key-value pairs?
6. What does `return` do in a function?
7. What type does `input()` return?
8. What error might `int('hello')` raise?
9. Why use `with` when opening a file?
10. What does `self` refer to in an instance method?

Answer key

1. It displays values in the output.
2. `=` assigns a value; `==` compares values.
3. 3.
4. The stop value is excluded.
5. A dictionary.

6. It sends a result back to the caller.
7. A string (str).
8. ValueError.
9. It helps ensure the file is properly closed.
10. The current object instance.

12. Four-week study plan

WEEK	FOCUS	PRACTICE	MILESTONE
1	Syntax, variables, types, input	Write 3 tiny programs daily	Tip calculator
2	Conditions, loops, collections	Solve 2 short problems daily	Guessing game
3	Functions, errors, files	Refactor code into functions	Quiz app
4	OOP and project building	Build and test one feature daily	Finish one project

Try a 45-minute session: 5 minutes recalling yesterday's topic, 15 minutes learning, 20 minutes coding, and 5 minutes writing a short summary. On busy days, do a 10-minute coding session rather than skipping.

13. Quick revision sheet

- Output: `print(value)`.
- Input: `input('Prompt: ');` convert with `int()` or `float()` if needed.
- Conditions: `if / elif / else;` use a colon and indentation.
- Loops: `for item in collection;` `while condition;` `range()` excludes its stop value.
- List: `append(value), items[index], len(items)`.
- Dictionary: `data[key];` safer optional lookup with `data.get(key, default)`.
- Function: `def name(parameters): ... return result`.
- Formatting: `f'Hello, {name}'`.
- Exceptions: `try / except SpecificError / else / finally`.
- Files: with `open(path, mode, encoding='utf-8')` as file: ...
- Modules: `import math; math.sqrt(25)`.
- OOP: `class Name; _init____` initializes an object; methods use `self`.

Glossary

Argument: A value passed to a function call.

Boolean: A logical value: True or False. **Expression:**

Code that evaluates to a value. **Iteration:** One pass through a loop.

Mutable: Can be changed after creation. **Parameter:**

A named input in a function definition. **Return value:**

The result sent back by a function.

Traceback: An error report showing the call path.

IDE: An application for writing, running, and debugging code.

Official resources

- Python tutorial: <https://docs.python.org/3/tutorial/>
- Python downloads: <https://www.python.org/downloads/>
- Standard library: <https://docs.python.org/3/library/>

Keep practising, testing, and explaining code in your own words. This guide is a starting point; use the official documentation for version-specific details.